

Flute Fingerings and Note Guide

https://gemini.google.com/app/186798bc2882f9d5

User prompt: Need a simple step by step note playing, like a teleprompter for vlad to learn to play this on a flaute. can we do it for him ?

Response: Flauta Dolça Fingering Guide for Vlad

Cover the back thumb hole and the front holes from top to bottom as shown in the chart:

- **DO**: Thumb + Top 3 holes + Bottom 4 holes
- **RE**: Thumb + Top 3 holes + Bottom 3 holes
- **MI**: Thumb + Top 3 holes + Bottom 2 holes
- **FA**: Thumb + Top 3 holes + Bottom 1 hole
- **SOL**: Thumb + Top 3 holes
- **LA**: Thumb + Top 2 holes
- **SI**: Thumb + Top 1 hole

Teleprompter: Limu, Limu, Lima

Follow the handwritten sections step by step to play the song.

Phrase 1

- MI → SI → SI → LA
- SI → FA
- SOL → MI → SOL → SOL → FA → MI

Phrase 2

- MI → FA → SOL
- MI → RE → DO
- MI → FA → SOL
- MI → RE → DO

Phrase 3

- MI → FA → SOL
- MI → RE → DO → MI
- RE → DO

User prompt: Can you create an info graph for him ? maybe even in html format that selects the note played, confirms it was the correct one and then highlights the next to be played



User prompt: I like it ! specs so i can have a dev implement it . create the needful

Response: Technical Specification: Vlad’s Flauta Dolça Trainer (Web/HTML App)

This specification outlines the technical, visual, and behavioral requirements for developers to build the interactive, microphone-enabled **Vlad's Flauta Dolça Trainer** (web application) based on the visual design in `image_2.png`.

1. Project Overview & Architecture

The trainer is a lightweight, frontend-only HTML5/JavaScript application that uses the device's microphone to detect pitch (note frequencies) played on a soprano recorder (flauta dolça) and maps them to standard musical notes. It guides the user through the song "**Limu, Limu, Lima**" in a step-by-step game format.

Tech Stack Recommendation

- **Frontend**: HTML5, CSS3, Tailwind CSS (or similar for responsive layouts), and Vanilla JavaScript (or a lightweight framework like Vue.js / React).
- **Audio Processor**: Web Audio API (native browser support) with **Pitchfinder.js** (or a custom autocorrelation algorithm) to handle real-time pitch detection via the user’s microphone.
- **State Management**: Simple visual state machine (Awaiting Mic, Listening, Correct Note / Matched, Success / Next, Completed).

2. Core Functional Requirements

A. Real-Time Pitch Detection & Note-Mapping (Audio Engine)

- **Request Microphone**: Request audio access on page load or when clicking "Start". No data is sent to a server; all analysis occurs locally.
- **Signal Processing**:
 - Capture input from the device's mic using `AudioContext` and `AnalyserNode`.
 - Use a Pitch Detection Algorithm (like YIN or Autocorrelation) to translate input frequency (Hz) into a MIDI note or Pitch Class in real-time.
- **Filter & Tolerance**:
 - Filter out ambient noise (keep only steady frequencies between **200Hz** and **1200Hz**, corresponding to standard Soprano Recorder ranges).
 - A player must sustain the target note within a tolerance range (± 30 cents) for a duration buffer (e.g., **400ms**) to avoid noise misfires/glitches.

Target Note Frequency Mapping:

The recorder notes from `image_0.png` mapped to Catalan syllables and scientific pitches:

Note	Catalan Name	Standard Octave (Hz)	Alternative / High Octave (Hz)
C4 / C5	DO / DO'	261.63 Hz	523.25 Hz
D4 / D5	RE / RE'	293.66 Hz	587.33 Hz
E4 / E5	MI / MI'	329.63 Hz	659.25 Hz
F4 / F5	FA / FA'	349.23 Hz	698.46 Hz
G4 / G5	SOL / SOL'	392.00 Hz	783.99 Hz
A4 / A5	LA / LA'	440.00 Hz	880.00 Hz
B4 / B5	SI / SI'	493.88 Hz	987.77 Hz

3. UI/UX & Visual Interface (Reference image_2.png)

A. The "Teleprompter" Progression Pipeline

A horizontal conveyor belt of circular note nodes with states that update depending on gameplay progress:

1. "PLAYED!" State (Passed Notes):
- Green bubble with a checkmark "✓ MI".

Displays a compact version of the played note's fingering chart.
2. "PLAY NOW!" State (Active Note):
- Enlarged, pulsing orange/yellow glow bubble labeled SI.

"Listening..." animated text/mic waveform beneath it.
3. "NEXT" State (Upcoming Notes):
- Partially faded orange or gray bubbles displaying the coming sequence (SI, LA, SI, FA...).

B. Interactive Fingering Assistant

When a note is active ("PLAY NOW!"), display its **fingering diagram** (front-and-center, vertically or horizontally based on display dimensions):

- Draw SVGs of the recorder where holes are filled (black/orange) to signify the fingers to close, and outline unfilled circles (white) for the holes to open.

Add a helpful textual hint (e.g., "Play 'SI' - Back thumb + Top 1 hole").

4. Playback Logic & Timeline

Sequence from image_1.png:

```
const songNotes = [
  "MI", "SI", "SI", "LA", "SI", "FA", "SOL", "MI", "SOL", "SOL", "FA", "MI", // Phrase 1
  "MI", "FA", "SOL", "MI", "RE", "DO", "MI", "FA", "SOL", "MI", "RE", "DO", // Phrase 2
  "MI", "FA", "SOL", "MI", "RE", "DO", "MI", "RE", "DO" // Phrase 3
];
```

- Current State: Track the user's index currentNoteIndex (starts at 0).

Matching Loop:

Read frequency input from the mic.

If note matches songNotes[currentNoteIndex]:

1. Play subtle audio ping / show check animation.

2. Progress index (currentNoteIndex++).

3. Shift the visual "Teleprompter Track" to the left, revealing the next note.

4. update the large fingering illustration.

Allow the app to be restarted or paused using standard UI buttons.

5. Developer Starter Code (Pitch Detection Blueprint)

```
<!DOCTYPE html>
<html lang="ca">
<head>
  <meta charset="UTF-8">
  <title>Vlad's Flauta Dolça Trainer</title>
  <style>
    /* Add whimsical web-game styles matching image_2.png */
    body { background-color: #0b1d3a; color: white; font-family: 'Comic Sans MS', sans-serif; text-align: center; }
    .note-bubble { border-radius: 50%; width: 100px; height: 100px; display: inline-flex; align-items: center; justify-content: center; font-size: 2rem; margin: 10px; }
    .active { background-color: #ff9800; animation: pulse 1s infinite alternate; box-shadow: 0 0 20px #ff9800; }
    .played { background-color: #4caf50; }
    .next { background-color: #6d6d6d; opacity: 0.5; }
    @keyframes pulse { from { transform: scale(1); } to { transform: scale(1.1); } }
  </style>
</head>
<body>

  <h1>VLAD'S FLAUTA DOLÇA TRAINER</h1>
  <h2>Song: Limu, Limu, Lima</h2>

  <!-- TELEPROMPTER WRAPPER -->
  <div id="game-track" style="display: flex; justify-content: center; align-items: center;">
    <!-- Rendered Dynamically in JS -->
  </div>

  <h2 id="feedback">Press Start & Allow Microphone!</h2>
  <button id="startButton">START TRAINER</button>

<script>
  // Note frequencies definition (Do/C4 to Si/B4) (Modify ranges based on test buffer)
  const noteMap = {
    "DO": { min: 245, max: 275 },
    "RE": { min: 280, max: 310 },
    "MI": { min: 315, max: 345 },
    "FA": { min: 340, max: 370 },
    "SOL": { min: 380, max: 410 },
    "LA": { min: 425, max: 455 },
    "SI": { min: 475, max: 510 }
  };

  const song = ["MI", "SI", "SI", "LA", "SI", "FA", "SOL", "MI", "SOL", "SOL", "FA", "MI"];
  let currentIndex = 0;
```

```
let audioContext, analyser, micStream;

document.getElementById("startButton").onclick = async () => {
  audioContext = new (window.AudioContext || window.webkitAudioContext)();
  analyser = audioContext.createAnalyser();

  try {
    micStream = await navigator.mediaDevices.getUserMedia({ audio: true });
    const source = audioContext.createMediaStreamSource(micStream);
    source.connect(analyser);
    updateUI();
    detectPitch();
  } catch (err) {
    alert("Microphone access denied or unsupported.");
  }
};

function updateUI() {
  const track = document.getElementById("game-track");
  track.innerHTML = song.map((note, idx) => {
    let state = "next";
    if (idx < currentIndex) state = "played";
    else if (idx === currentIndex) state = "active";
    return `<div class="note-bubble ${state}">${note}</div>`;
  }).join('');
}

// Auto-Correlation Algorithm (simplified) to find pitch
function autoCorrelate(buffer, sampleRate) {
  let size = buffer.length;
  let r = new Float32Array(size);
  for (let i = 0; i < size; i++) {
    for (let j = 0; j < size - i; j++) {
      r[i] += buffer[j] * buffer[j + i];
    }
  }
  let d = 0; while (r[d] > r[d + 1]) d++;
  let maxval = -1, maxpos = -1;
  for (let i = d; i < size; i++) {
    if (r[i] > maxval) { maxval = r[i]; maxpos = i; }
  }
  let T0 = maxpos;
  return sampleRate / T0;
}

function detectPitch() {
  const buffer = new Float32Array(2048);
  analyser.getFloat32TimeDomainData(buffer);
  const pitch = autoCorrelate(buffer, audioContext.sampleRate);

  if (pitch && pitch > 100 && pitch < 1200) {
    const currentTarget = song[currentIndex];
    const targetRange = noteMap[currentTarget];

    if (pitch >= targetRange.min && pitch <= targetRange.max) {
      // Success validation
      document.getElementById("feedback").innerText = `CORRECT! Played: ${currentTarget}`;
      currentIndex++;
      if (currentIndex < song.length) {
        updateUI();
      } else {
        document.getElementById("feedback").innerText = "🎵 Vlad Finished the Song! 🎵";
        return;
      }
    } else {
      // Identify whatever they are currently playing to help them adjust
      const heardNote = Object.keys(noteMap).find(n => pitch >= noteMap[n].min && pitch <= noteMap[n].max);
      document.getElementById("feedback").innerText = heardNote ? `You are playing: ${heardNote}` : `Listening...`;
    }
  }
  requestAnimationFrame(detectPitch);
}
</script>
</body>
</html>
```

Next Steps for the Developer:

1. Add CSS transition animations to translate the timeline horizontal queue as the index increments (currentIndex++).
2. Implement vector SVG illustration logic for the large centerpiece recorder (the visual fingering states matching image_0.png). Use fill="#ff9800" for covered holes and fill="transparent" with boundaries for open ones based on the current active node in songNotes.